ALL REPO LINKS-

https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction

Data collection-

https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/Data%20collection%20spacex.ipynb

WEB Scrapping –

https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/spacex-webscraping.ipynb

Data wrangling-

https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/data%20wrangling.ipynb

EDA with SQL-
https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/Eda%20%20with%20SQL.ipynb

EDA With Datavisualization-

https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/EDA%20WITH%20DATA%20VISUALIZATION.ipynb

SpaceX launch site-

https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/Spacex%20Launch%20site.ipynb

Interactive Dash App-
https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/spacex-dash-app.py

Machine Learning Predictive Analysis-
https://github.com/mamta3925-cse/SpaceX-Falcon-9-Landing-Prediction/blob/main/SpaceX-Machine-Learning-Prediction.ipynb

APPLIED DATA SCIENCE CAPSTONE

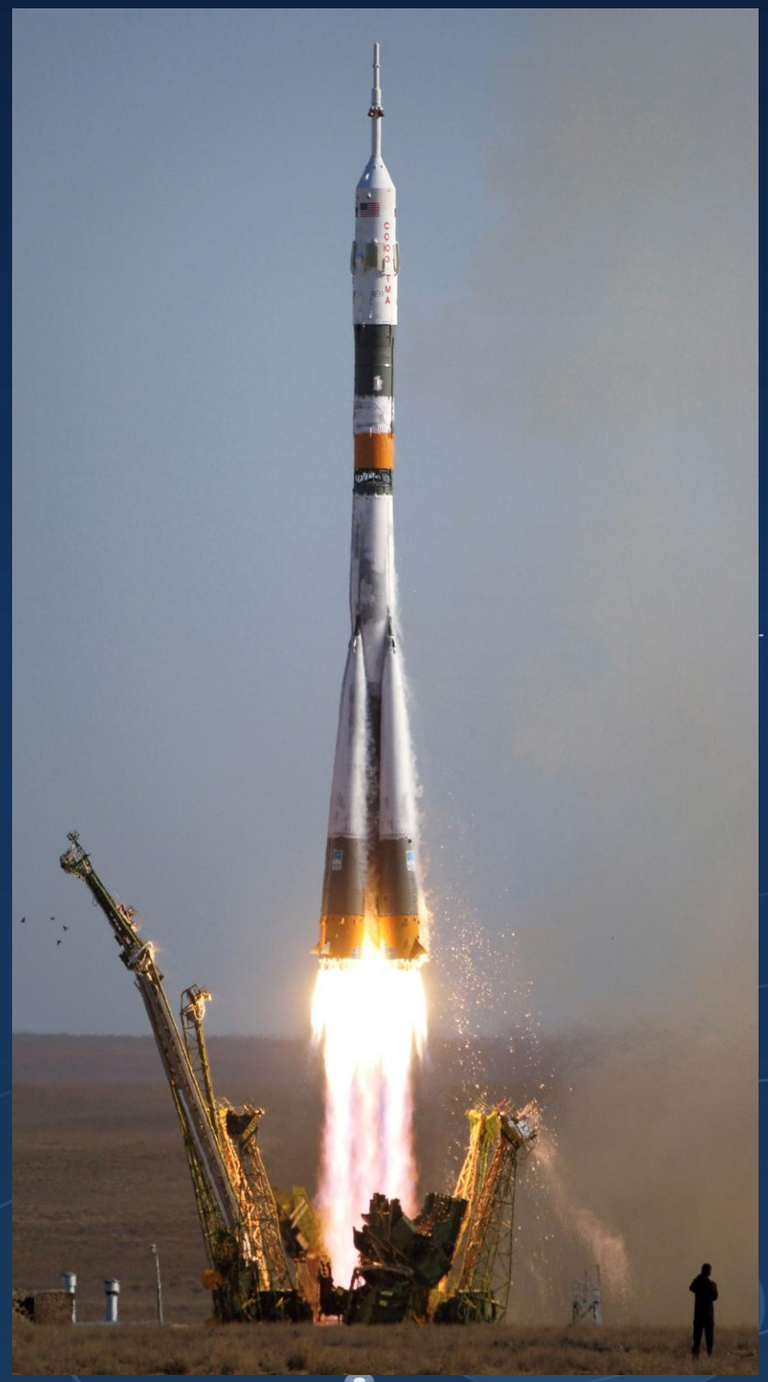
Predicting Falcon 9 First-Stage Landing Outcomes

A Data-Driven Bidding Strategy for a Competing Launch Provider

Prepared by:
[Mamta Sahu]
IBM Data Science Professional Certificate
[August2026]

OUTLINE

- **Executive Summary**
- Introduction
- Methodology
- Result
- Conclusion
- Appendix



Executive Summary

➢Summary of Methodology

-Data Collection

-Data Wrangling

-Eda with SQL

Building an interactive map with folium

Building a Dashboard with Plotly Dash

Predictive Analysis(Classification)

➢ Summary OF all results

-Exploratory data analysis results

-Iinteractive Dashboard

-Predictive analysis result

Introduction

- ***Project Background and Context***

We aimed to forecast whether the Falcon 9 first stage

would touch down successfully.

SpaceX markets its Falcon 9 launches at a price of 62 million dollars, while competing providers charge upward of 165 million dollars per launch — a difference driven largely by SpaceX's ability to reuse the first stage. So, by figuring out whether the first stage will land, we can estimate the overall launch cost. This insight becomes valuable for a rival company looking to compete with SpaceX for a launch contract.

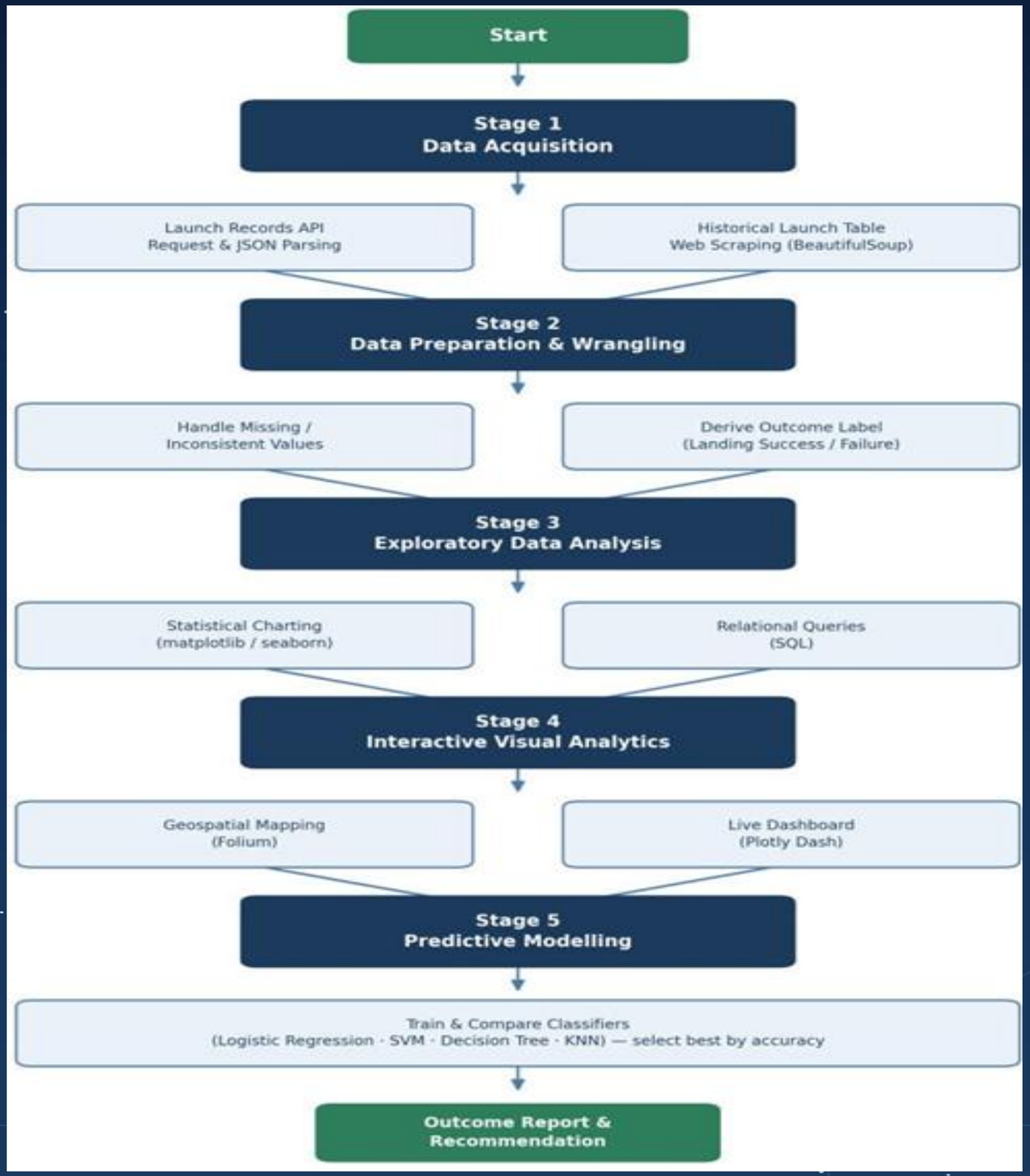
- ***Problems that needed addressing:-***

Which factors affect a rocket's chances of landing successfully?

How much impact does each rocket-related variable have on the probability of a successful landing?

What conditions must SpaceX meet to maximize the likelihood of a successful landing?

Methodology



- **Data collection methodology**:-
  Gathered data through the SpaceX REST API and web scraping from Wikipedia

  •Cleaned and reshaped the data to prepare it for machine learning (data wrangling)

  •Applied One Hot Encoding to categorical fields and removed columns that weren't useful

  •Explored the data (EDA) using SQL queries and visual plots — scatter and bar charts to spot trends and relationships between variables

  •Built interactive visualizations using Folium and Plotly Dash

  •Ran predictive analysis with classification models — building, fine-tuning, and evaluating them for accuracy

[github]

Methodology

Data collection – SpaceX API

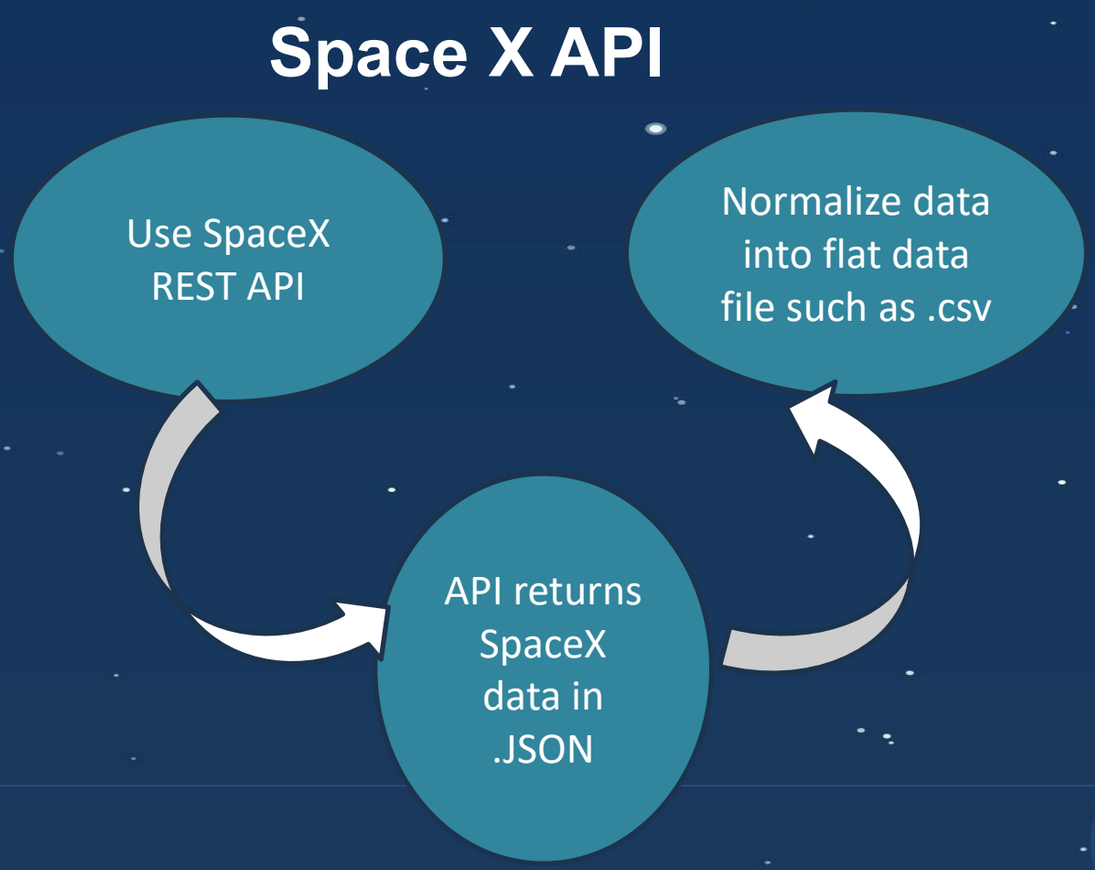
The following datasets were collected in this way:
We used the SpaceX REST API to gather SpaceX launch data.

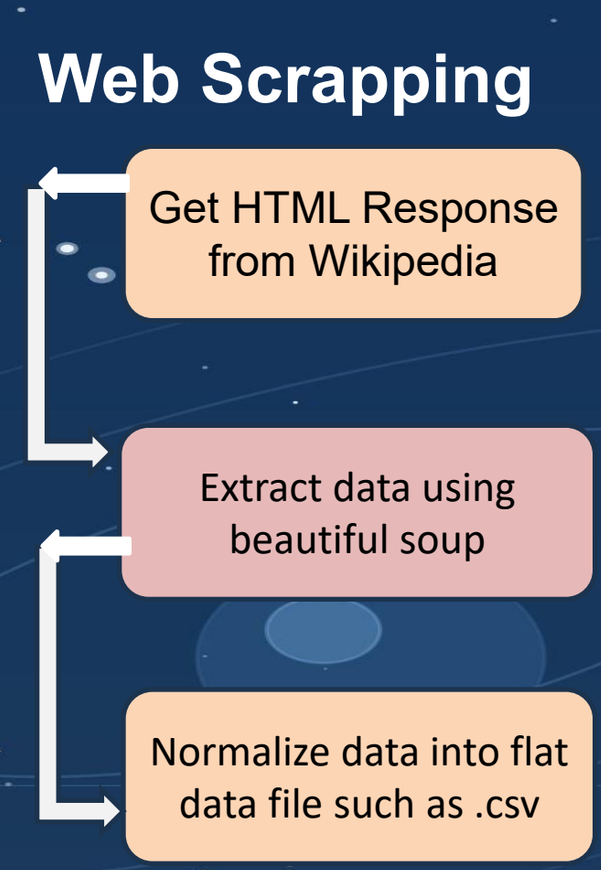
This API provides information about the rocket, payload, launch details, and landing outcome.

Our goal is to predict whether SpaceX will attempt to land the rocket or not.

The SpaceX REST API endpoint starts with api.spacexdata.com/v4/.

Another source of Falcon 9 launch data is web scraping from Wikipedia using BeautifulSoup.





1. Getting Response From API

```
spacex_url = "https://api.spacexdata.com/v4/launches/past"
response = requests.get(spacex_url).json()
```

2. Convert response to JSOn

```
response = request.get(static_json_url).json()
data = pd.json_normalize(response.json())
```

3. Apply custom cleaning functions

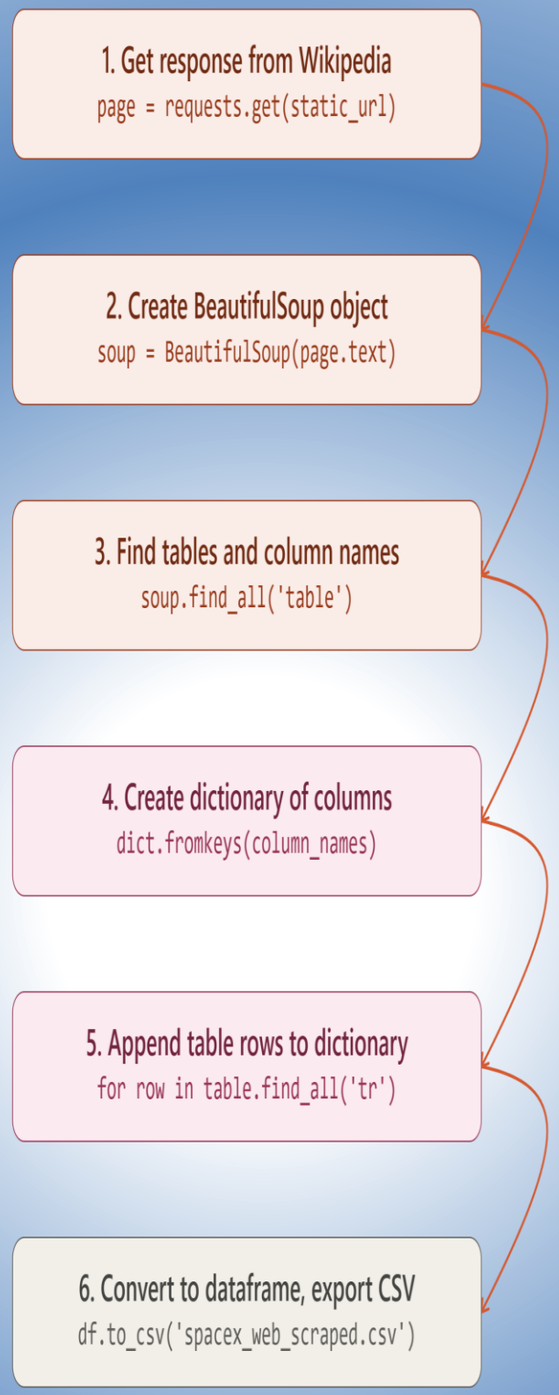
```
getBoosterVersion(data)
getLaunchSite(data)
getPayloadData(data)
getCoreData(data)
```

4. Build dictionary, then data

```
launch_dict = {  'FlightNumber': list(data['flight_number ])
'Date': list(data['date’])
 'BoosterVersion': BoosterVersion ,
‘ PayloadMass': PayloadMas ,
 'Orbit': Orbit,
'LaunchSite': Launch,
 'Outcome': Outcome,
 'Flights': Flight,
 'GridFins': GridFins
 'Reused’: Reused,
 'Legs': Legs,
 'LandingPad': LandingPad,
 ‘Block': Block,
 'ReusedCount': ReusedCoun,
 'Serial': Serial,
'Longitude': Longitude,
'Latitude’: Latitude
df = pd.DataFrame.from_dict(launch_dict
```

5. Filter and export

```
data_falcon9 = df.loc[df['BoosterVersion'] == 'Fa
data_falcon9.to_csv('dataset_part_1.csv', index=False
```



1. Get response from Wikipedia

```
page = requests.get(static_url)
```

2. Create BeautifulSoup object

```
soup = BeautifulSoup(page.text, 'html.parser')
```

3. Find tables

```
html_tables = soup.find_all('table')
```

4. Extract column names

```
column_names = []
temp = soup.find_all('th')
for x in range(len(temp)):
    try:
        name = extract_column_from_header(temp[x])
        if name is not None and len(name) > 0:
            column_names.append(name)
    except:
        pass
```

5. Create dictionary and append rows

```
launch_dict = dict.fromkeys(column_names)
del launch_dict['Date and time ( )']
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
launch_dict['Version Booster'] = []
launch_dict['Booster landing'] = []
launch_dict['Date'] = []
launch_dict['Time'] = []
```

```
extracted_row = 0
for table_number, table in enumerate(soup.find_all('table')):
for rows in table.find_all('tr'):
# extract each row's data and append to launch_dict
```

6. Convert to dataframe and export

```
df = pd.DataFrame.from_dict(launch_dict)
df.to_csv('spacex_web_scraped.csv', index=False)
```

Data Wrangling

Introduction

In the dataset, there are several cases where the booster failed to land successfully. Sometimes a landing was attempted but didn't work out due to an accident. For example, True Ocean means the mission successfully landed in a specific ocean region, while False Ocean means it failed to land there. True RTLS means the mission successfully landed on a ground pad, while False RTLS means it failed to land on a ground pad. True ASDS means the mission successfully landed on a drone ship, while False ASDS means it failed to land on a drone ship. We convert these outcomes into training labels — 1 means the booster landed successfully, and 0 means it did not.

Perform Explorartory Data Analysis EDA on dataset

Calculate the number of launches at each site

Calculate the number and occurrence of each orbit

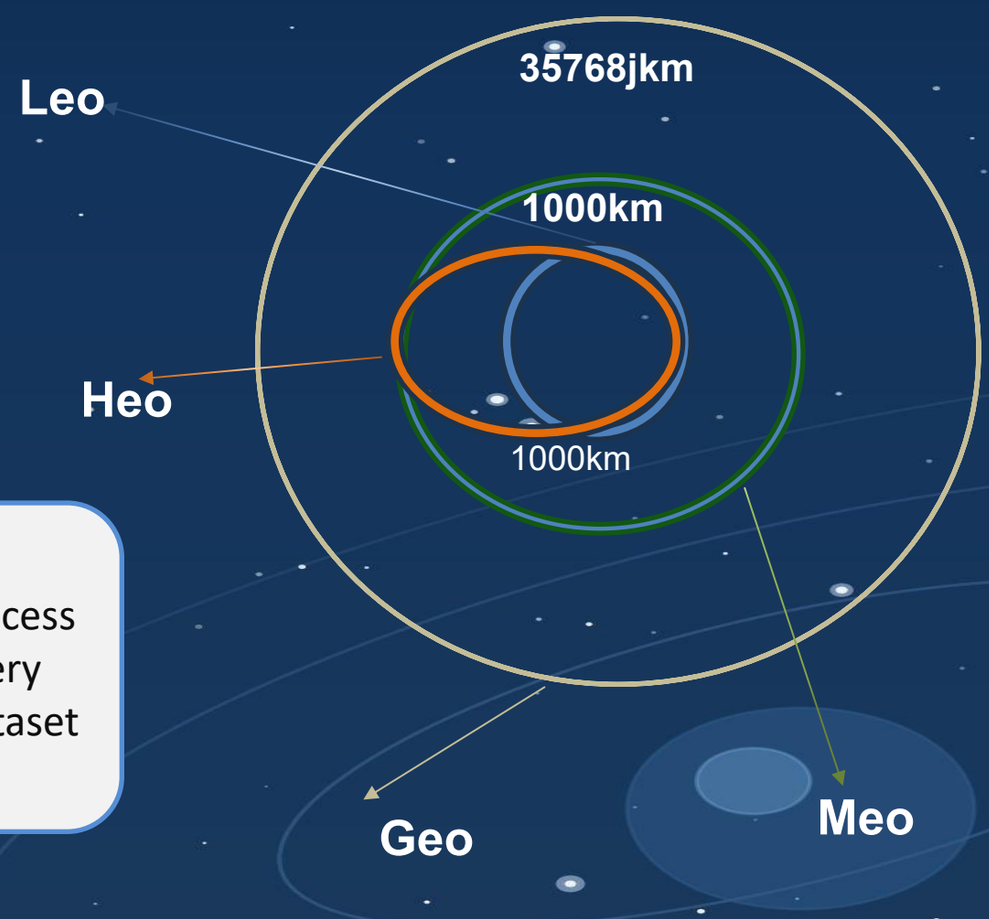
Calculate the number and occurrence of mission outcome per orbit type

Export dataset as .CSV

Create a landing outcome label from Outcome column

Work out success rate for every landing in dataset

Each launch aims to an dedicated orbit, and here are some common orbit types



Dagram common orbit type spaceX usws

EDA With Data Visualization

Scatter Graphs being drawn:

Flight Number VS. Payload Mass

Flight Number VS. Launch Site

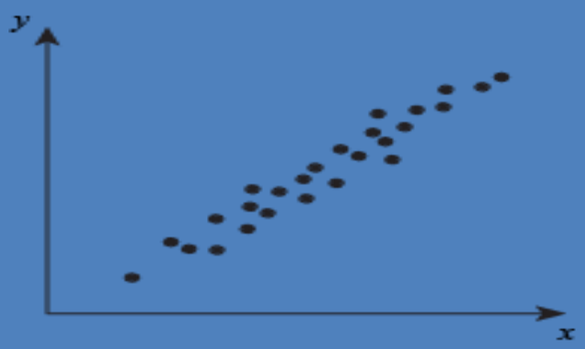
Payload VS. Launch Site

Orbit VS. Flight Number

Payload VS. Orbit Type

Orbit VS. Payload Mass

Scatter plots show how much one variable is affected by another. The relationship between two variables is called their correlation . Scatter plots usually consist of a large body of data. GitHub URL to Notebook.



Bar Graph being drawn:

Mean VS. Orbit

A bar diagram makes it easy to compare sets of data between different groups at a glance.
The graph represents categories on one axis and a discrete value in the other. The goal is to
show the relationship between the two axes. Bar charts can also show big changes in data over time.

Line Graph being drawn:

Success Rate VS. Year

Line graphs are useful in that they show data variables and trends very clearly and can help
to make predictions about the results of data not yet recorded

EDA With SQL

We ran SQL queries to explore and understand the dataset. Below are some of the questions we needed answers to, along with the queries used to find them in the data:-

- Show the names of unique launch sites used in space missions.
- Show 5 records where the launch site name starts with 'KSC'.
- Show the total payload mass carried by boosters launched under NASA (CRS).
- Show the average payload mass carried by the F9 v1.1 booster version.
- List the dates when a successful drone ship landing was achieved.
- List the names of boosters that landed successfully on a ground pad and carried a payload mass between 4000 and 6000.
- List the total count of successful and failed mission outcomes.
- List the booster versions that carried the highest payload mass.
- List the month names, successful ground pad landings, booster versions, and launch sites for the year 2017.
- Rank the count of successful landing outcomes between 2010-06-04 and 2017-03-20 in descending order.

Building an interactive map with Folium

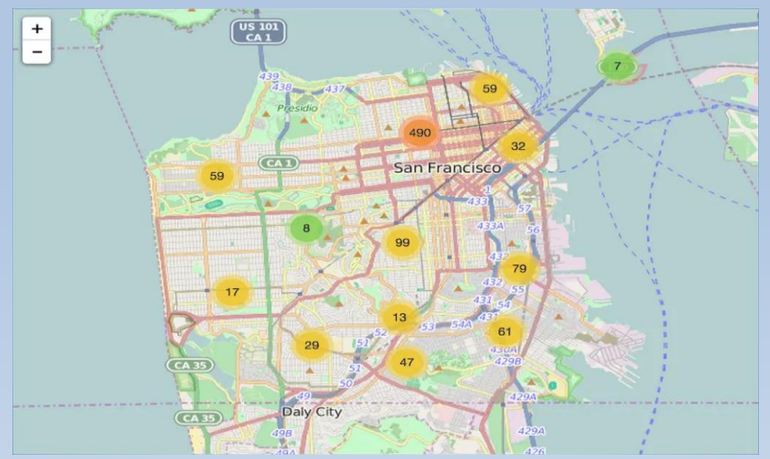
To visualize the launch data on an interactive map, we used the latitude and longitude of each launch site and placed a circle marker around each one, labeled with the launch site's name.

We grouped the launch outcomes (failures and successes) into classes 0 and 1, showing them as red and green markers inside a MarkerCluster() on the map.

Using the Haversine formula, we calculated the distance from each launch site to nearby landmarks to identify patterns in what surrounds each site. Lines were drawn on the map to show these distances to landmarks.

Some example trends we found about where launch sites are located:

- Are launch sites close to railways?
- NoAre launch sites close to highways**?**
- NoAre launch sites close to the coastline? Yes
- Do launch sites keep a certain distance from cities? Yes



Building an interactive map with Folium

Used Python Anywhere to host the website live 24/7 so your can play around with the data and view the data-

The dashboard is built with Flask and Dash web framework.
Graphs

-Pie Chart showing the total launches by a certain site/all
Sites

- display relative proportions of multiple classes of data.- size of the circle can be made proportional to the total quantity it represents.

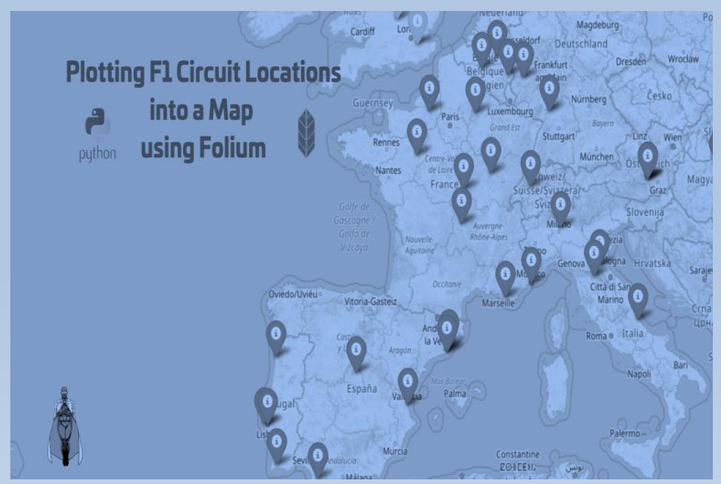
Scatter Graph showing the relationship with Outcome and Payload Mass (Kg) for the different Booster Versions

- It shows the relationship between two variables

.- It is the best method to show you a non-linear pattern

.- The range of data flow, i.e. maximum and minimum value, can be determined.

- Observation and reading are straightforward.



Predictive analysis (Classification)

BUILDING MODEL-

Load the dataset into NumPy and Pandas.

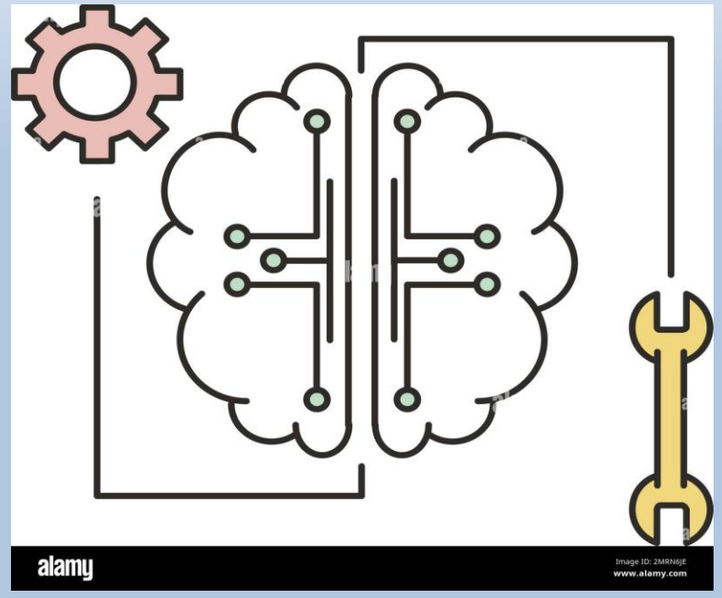
Transform the dataSplit the data into training and test sets.

Check the number of test samples we have.

Choose which machine learning algorithms to use.

Set the parameters and algorithms in GridSearchCV.

Fit the dataset into the GridSearchCV objects and train the model.



EVALUATING MODEL-

Check the accuracy of each mode

IGet the tuned hyperparameters for each algorithm type

Plot the confusion matrixl

MPROVING MODEL-

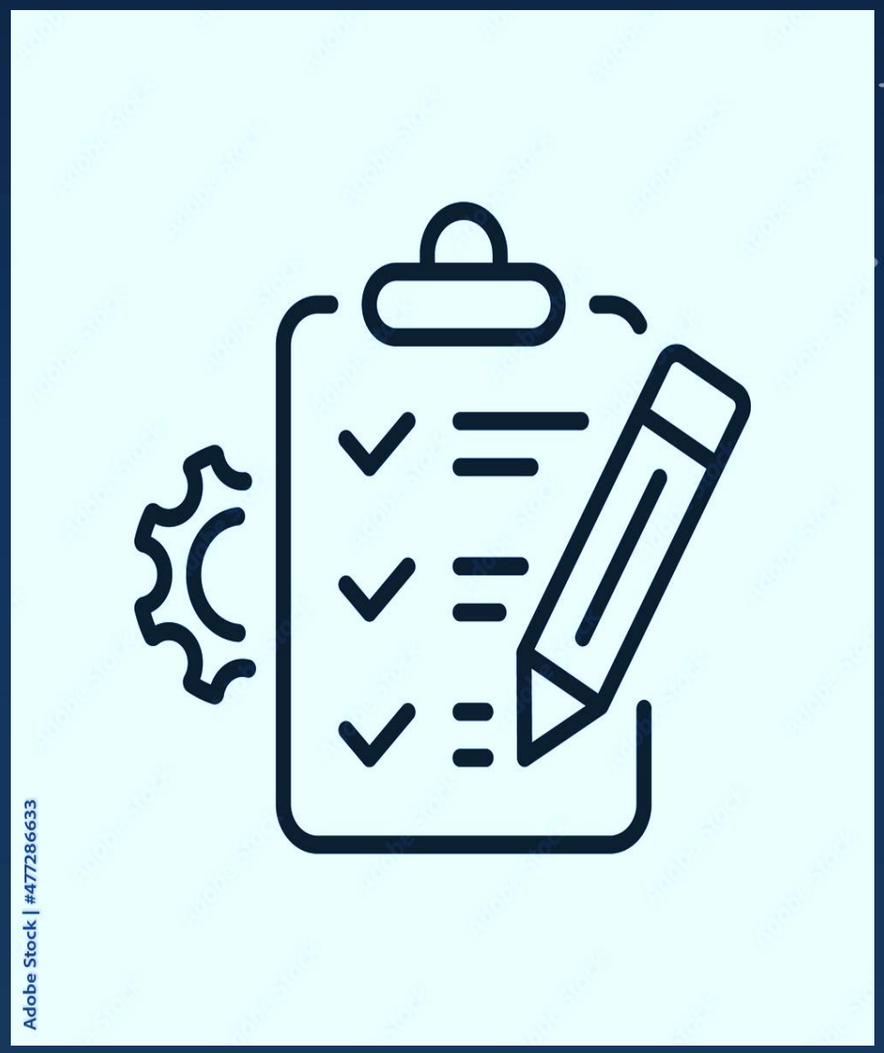
Feature engineering

Algorithm tuning

FINDING THE BEST PERFORMING CLASSIFICATION MODEL-

The model with the highest accuracy score is selected as the best performing mode

RESULT



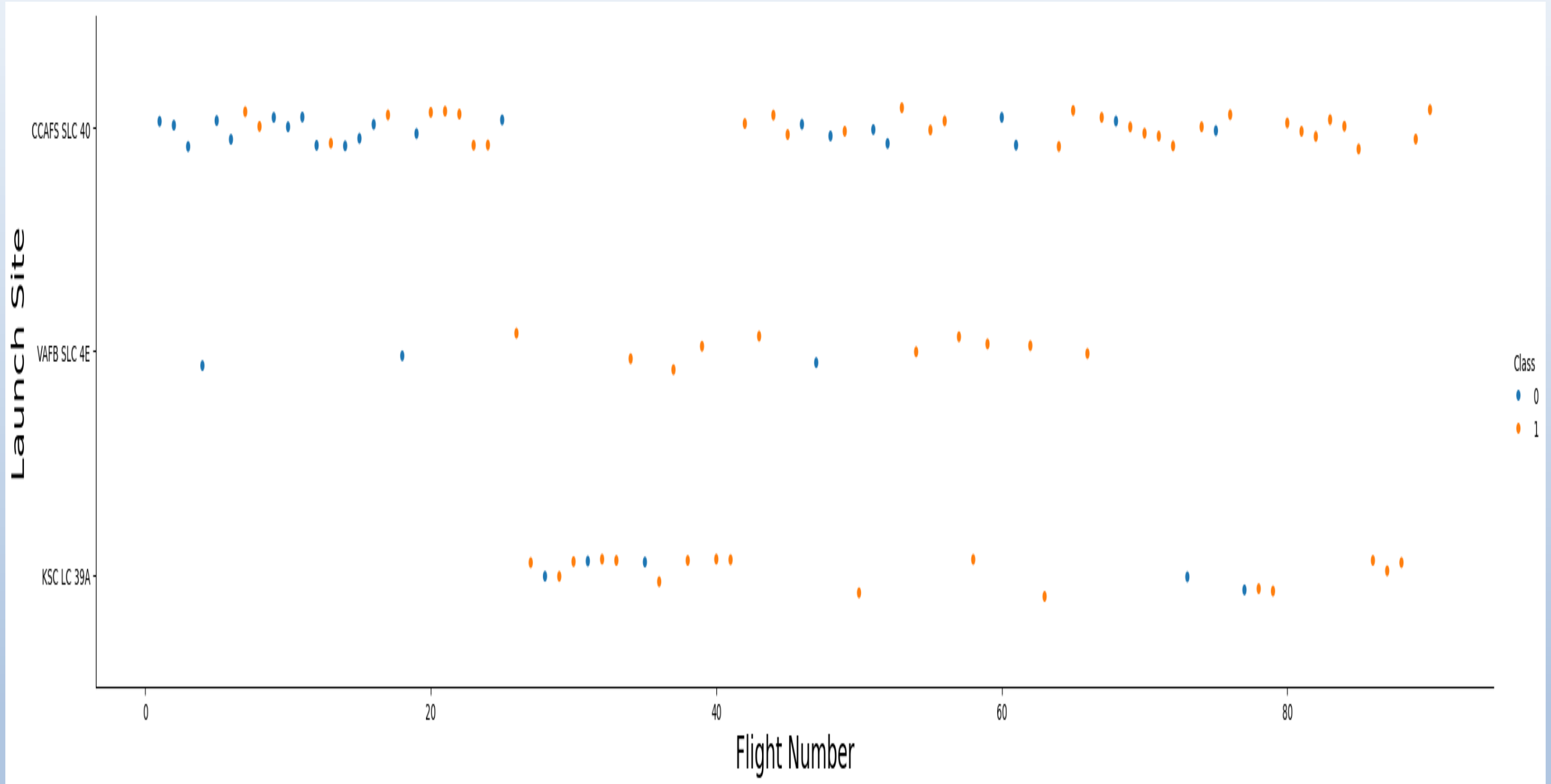
Exploratory data analysis results

- Interactive analytics demo in screenshots

- Predictive analysis results

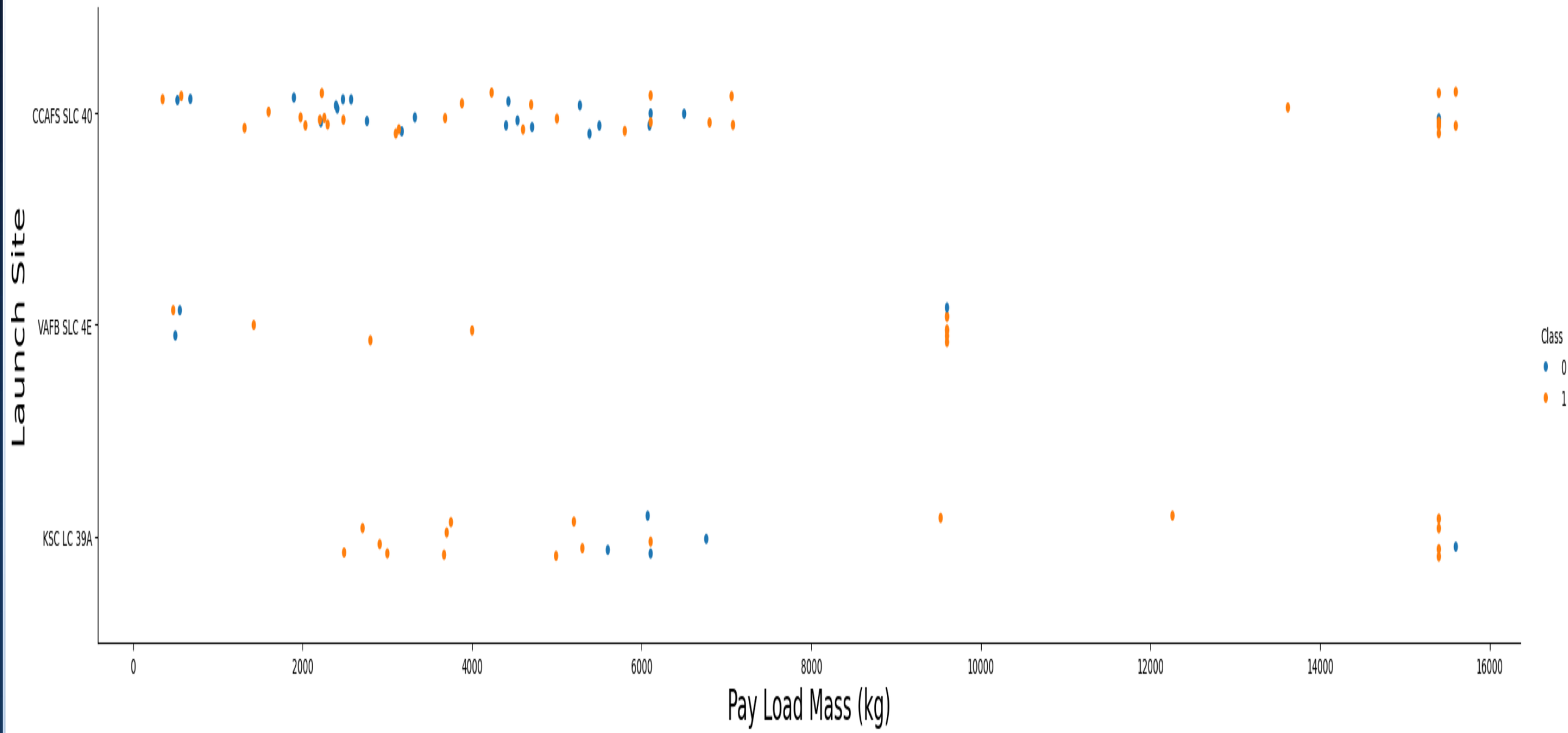
EDA With Visualization

Flight Number vs. Launch Site



The more amount of flights at a launch site the greater the success rate at a launch sit

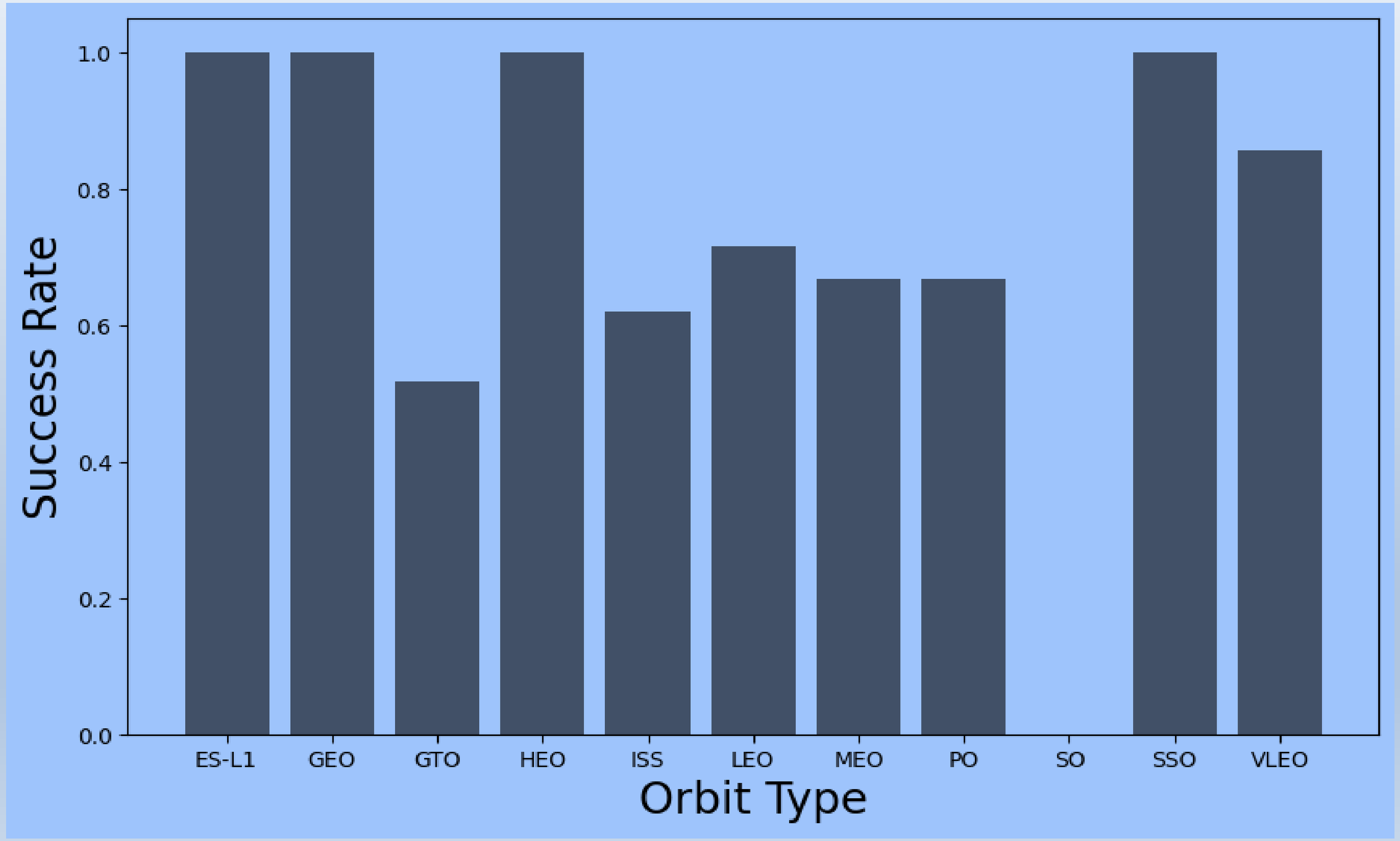
Payload Mass vs. Launch site



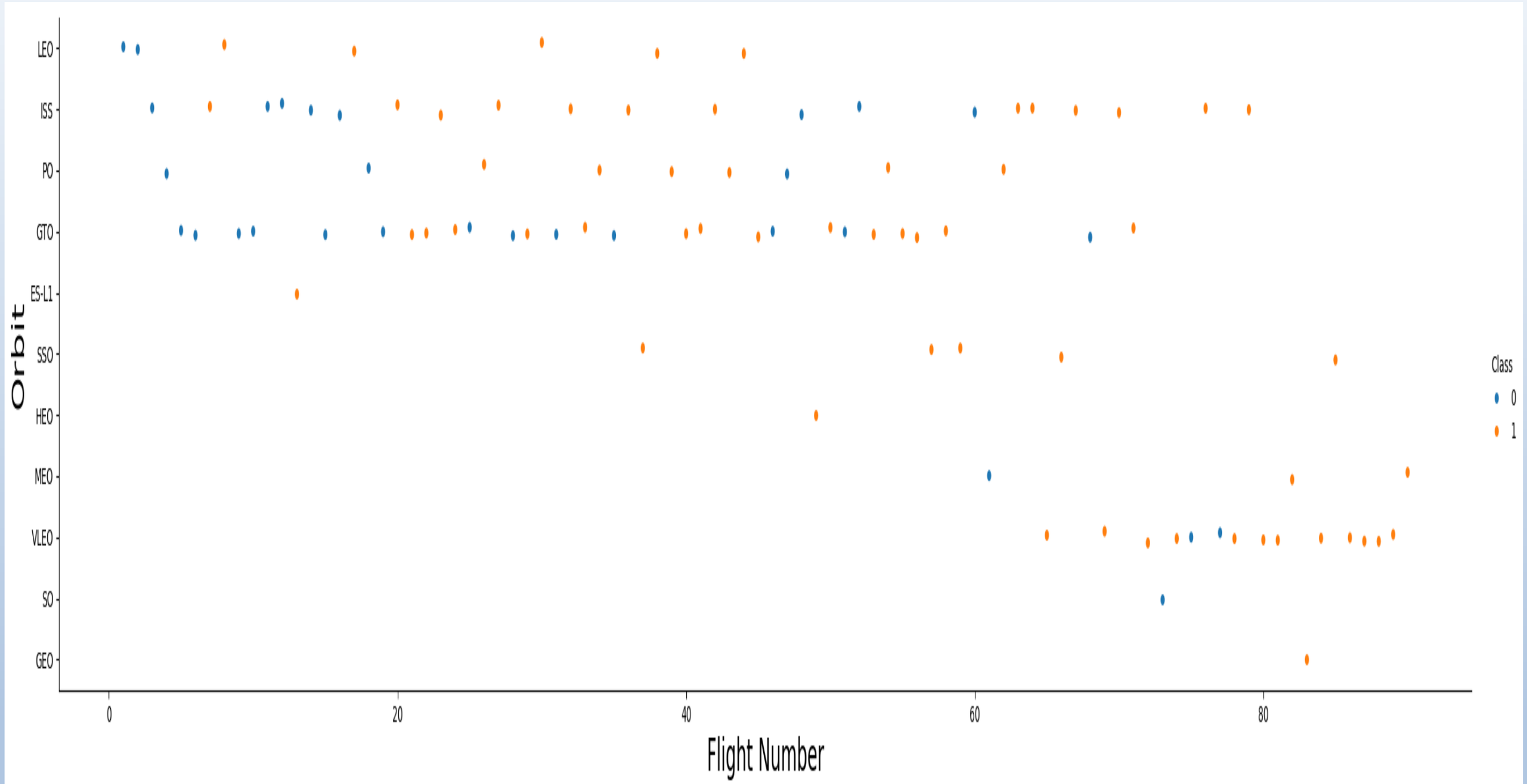
The greater the payload mass for Launch Site CCAFS SLC 40 the higher the success rate for the Rocket. There is not quite a clear pattern to be found using this visualization to make a decision if the Launch Site is dependant on Pay Load Mass for a success launch.

Success rate vs. Orbit type

Orbit GEO, SSO, HEO, ES-L1 has the best success Rate

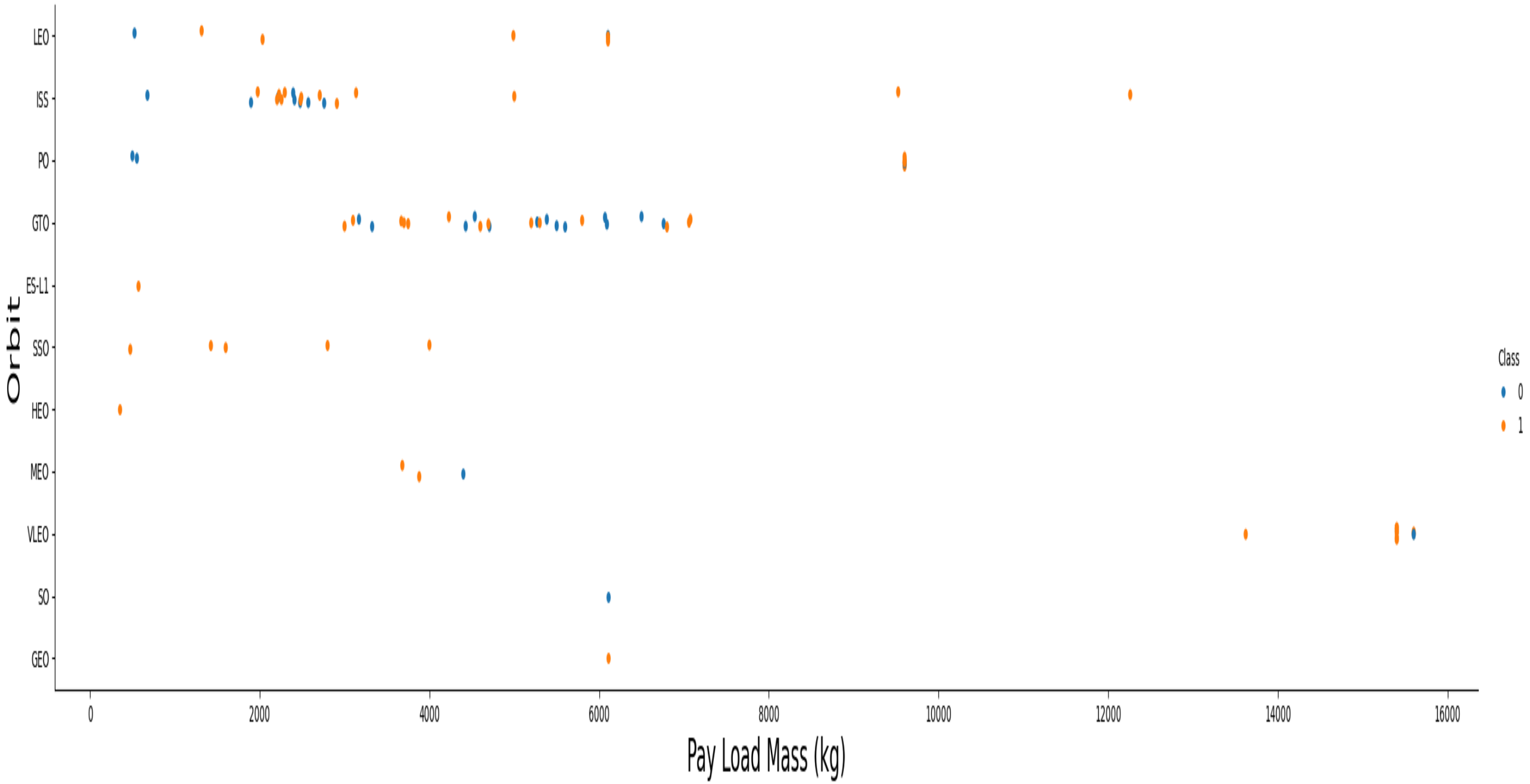


Flight Number vs. Orbit type



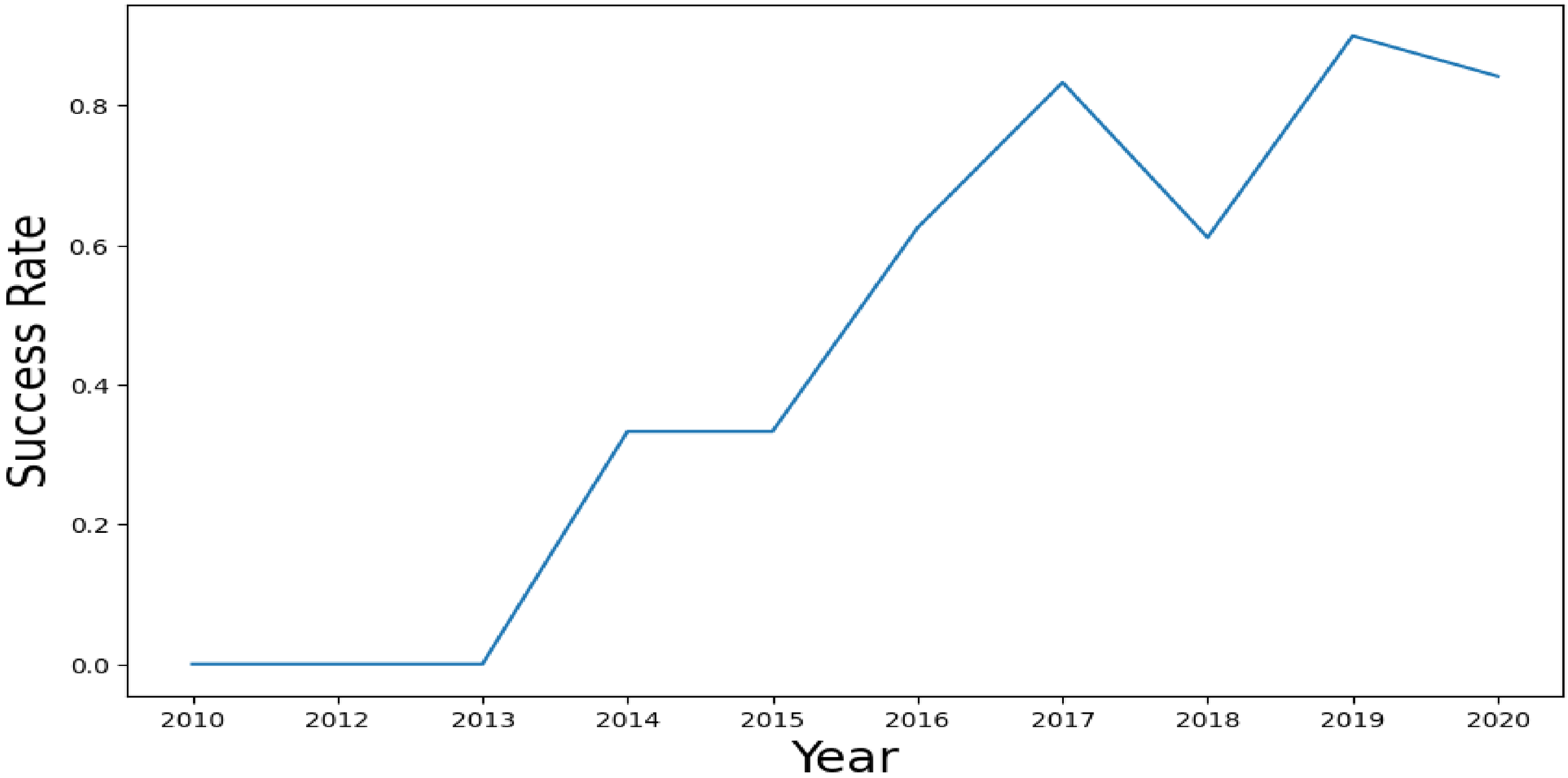
You should see that in the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit

Payload vs. Orbit type



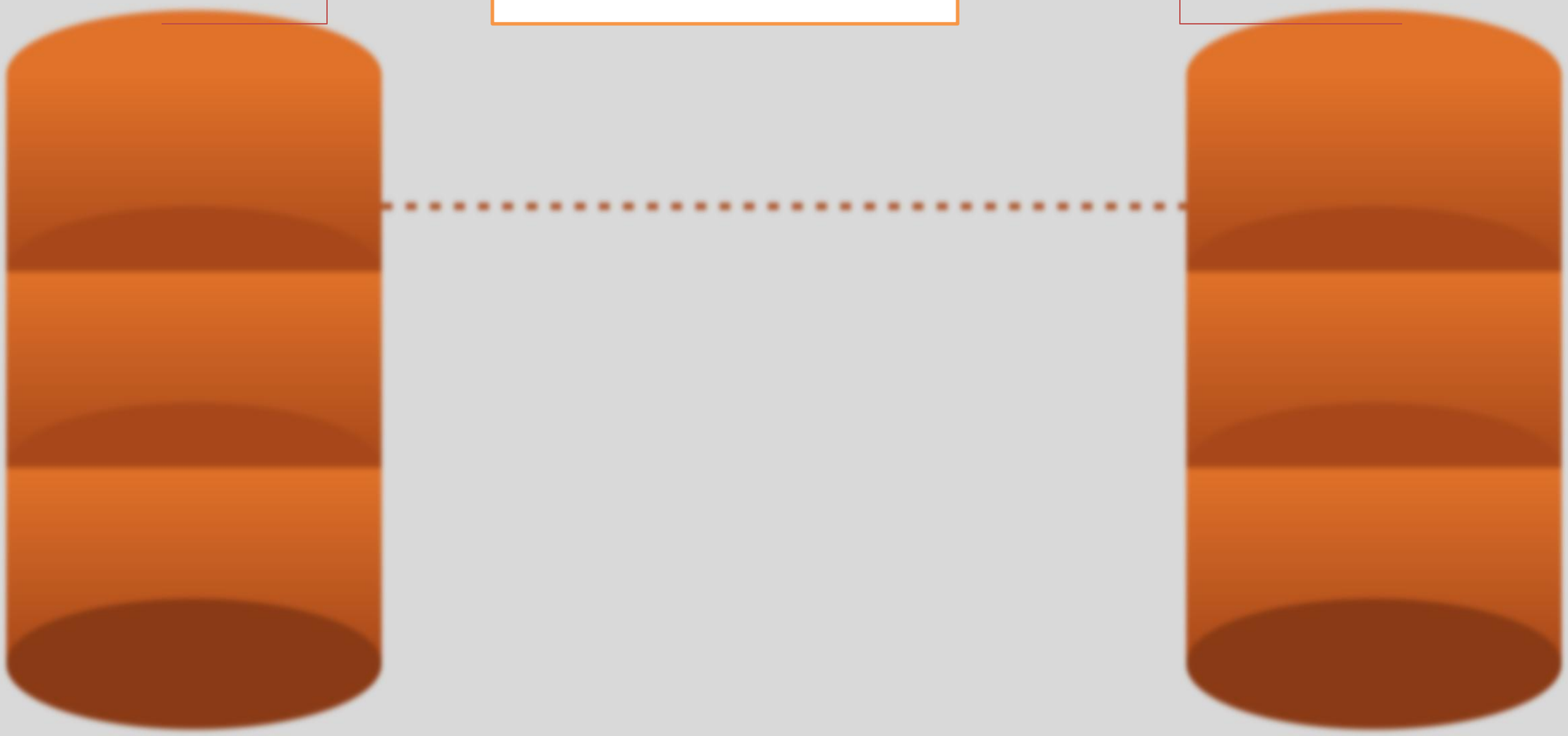
You should observe that Heavy payloads have a negative influence on GTO orbits and positive on GTO and Polar LEO (ISS) orbits.

Launch success Yearly Trend



you can observe that the success rate since 2013 kept increasing till 2020

EDA WITH .SQL



Unique Launch Sites

SQL QUERY

select DISTINCT Launch_Site from tblSpaceX

Unique Launch Site
CCAFS LC-40
CCAFS SLC-40
KSC LC -39A
VAFB SLC -4E

QUERY EXPLAINATION

Using the word DISTINCT in the query means that it will only show Unique values in the Launch_Site column from tblSpaceX

Launch Site names begin with CCA

SQL QUERY

SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5*;*

QUERY EXPLAINATION

Using the LIKE 'CCA%' keyword has a wildcard with the letters 'CCA', where the percentage sign at the end suggests that the Launch_Site name must start with CCA, and LIMIT 5 restricts the output to only 5 records.

Date	Time (utc)	Booster_version	LAUNCH SITE	PAYLOAD	Payload_mass_kg	orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2013-03-01	0:35:00	F9 v1.0 B0006	CCAFS LC-40	Dragon demo flight C2	500	LEO (ISS)	NASA (crs)	Success	No attempt
2012-10-08	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-1	677	LEO (ISS)	NASA (crs)	Success	No attempt

TOTAL PAYLOAD MASS BY CUSTOMER NASA (CRS)

SQL QUERY

```
%%sql SELECT SUM("PAYLOAD_MASS__KG_") AS "Total_Payload_Mass_KG" FROM SPACEXTBL WHERE "Customer" = 'NASA (CRS)';
```

TOTAL PAYLOAD MASS
45595

QUERY EXPLAINATION

Using the SUM() function adds up all values in the "PAYLOAD_MASS__KG_" column, and the WHERE "Customer" = 'NASA (CRS)' clause restricts this calculation to only rows where the customer is NASA (CRS).

Average Payload Mass carried by booster version F9 v1.1

SQL QUERY

```
%%sql
SELECT AVG("PAYLOAD_MASS__KG_") AS "Average_Payload_Mass_KG
FROM SPACEXTBL
WHERE "Booster_Ver
```

Average_Payload_Mass_KG
2928

QUERY EXPLAINATION

Using the AVG() function calculates the mean value of the "PAYLOAD_MASS__KG_" column across all matching records in the table.

```
sion" = 'F9 v1.1';
```

The date where the successful landing outcome in drone ship was achieved First successful gro

SQL QUERY

```
 %%sql
SELECT MIN("Date") AS "First_Successful_Ground_Pad_Landing"
FROM SPACEXTBL
WHERE "Landing_Outcome" = 'Success (ground pad)';
```

First_Successful_Ground_Pad_Landing
2015-12-22

QUERY EXPLAINATION

Using the MIN("Date") function returns the earliest date value, and the WHERE clause filters records to only those where Landing_Outcome exactly matches 'Success (ground pad)'

Total Number of Successful and Failure Mission Outcomes

SQL QUERY

```
%%sql

SELECT
    (SELECT COUNT(Mission_Outcome) FROM SPACEXTBL WHERE Mission_Outcome LIKE '%Success%') AS
Successful_Mission_Outcomes,
    (SELECT COUNT(Mission_Outcome) FROM SPACEXTBL WHERE Mission_Outcome LIKE '%Failure%') AS
Failed_Mission_Outcomes
```

Successful_Mission_Outcomes	Failed_Mission_Outcomes
100	**1**

QUERY EXPLAINATION

Using two separate subqueries with LIKE '%Success%' and LIKE '%Failure%' counts how many Mission_Outcome records contain the words "Success" and "Failure" respectively, displaying both totals side by side in one row.

Boosters carried maximum payload

SQL QUERY

%%sql

SELECT DISTINCT Booster_Version, MAX(PAYLOAD_MASS__KG_) AS "Maximum Payload Mass"FROM SPACEXTBLGROUP BY Booster_VersionORDER BY "Maximum Payload Mass" DESC

Booster_version	Maximum payload mass
F9 B5 B1060.3	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1056.4	15600
F9 B5 B1051.6	15600
F9 B5 B1051.4	15600
F9 v1.0 B0005	525
F9 v1.0 B0003	500
F9 v1.0 B0006	500
97 rows * 2 col	

Using GROUP BY Booster_Version groups records by each distinct booster, and the MAX(PAYLOAD_MASS__KG_) function finds the heaviest payload carried by each one, sorted highest to lowest with ORDER BY ... DESC.

2017 Launch Records

SQL QUERY

%%sq

ISELECT CASE CAST(strftime('%m', Date) AS INTEGER)
WHEN 1 THEN 'January' WHEN 2 THEN 'February' WHEN 3 THEN 'March' WHEN 4 THEN 'April' WHEN 5 THEN 'May'
WHEN 6 THEN 'June' WHEN 7 THEN 'July' WHEN 8 THEN 'August' WHEN 9 THEN 'September'
 WHEN 10 THEN 'October' WHEN 11 THEN 'November' WHEN 12 THEN 'December' END AS Month, B
ooster_Version, Launch_Site,Landing_OutcomeFROM SPACEXTBLWHERE Landing_Outcome LIKE '%Success%'AND
strftime('%Y', Date) = '2017'

Month	Boosterversion	Launch_Site	Landing_Outcome
januar	F9 FT B1029.1	VAFB SLC-4E	Success (drone ship)
February	F9 FT B1031.1	KSC LC-39A	Success (ground pad)
March	F9 FT B1021.2	KSC LC-39A	Success (drone ship)
May	F9 FT B1032.1	KSC LC-39A	Success (ground pad)
June	F9 FT B1035.1	KSC LC-39A	Success (ground pad)
June	F9 FT B1029.2	KSC LC-39A	Success (drone ship)
June	F9 FT B1036.1	VAFB SLC-4E	Success (drone ship)
August	F9 B4 B1039.1	KSC LC-39A	Success (ground pad
August	F9 FT B1038.1	VAFB SLC-4E	Success (drone ship)
September	F9 B4 B1040.1	KSC LC-39A	Success (ground pad)
October	F9 B4 B1041.1	VAFB SLC-4E	Success (drone ship)
December	F9 FT B1035.2	CCAFS SLC-40	Success (ground pad)
October	F9 FT B1031.2	KSC LC-39A	Success (drone ship)
October	F9 B4 B1042.1	KSC LC-39A	Success (drone ship)

QUERY EXPLAINATION

Using LIKE '%Success%' filters records where the Landing_Outcome contains the word "Success" anywhere in the text, and strftime('%Y', Date) = '2017' restricts results to only the year 2017, while the CASE statement converts the numeric month into its full name.

Rank success count between 2010-06-04 and 2017-03-20

SQL QUERY

%%sqlSELECT "Landing_Outcome", COUNT(*) AS "Outcome_Count"FROM SPACEXTBLWHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20'GROUP BY "Landing_Outcome"ORDER BY "Outcome_Count" DESC;

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

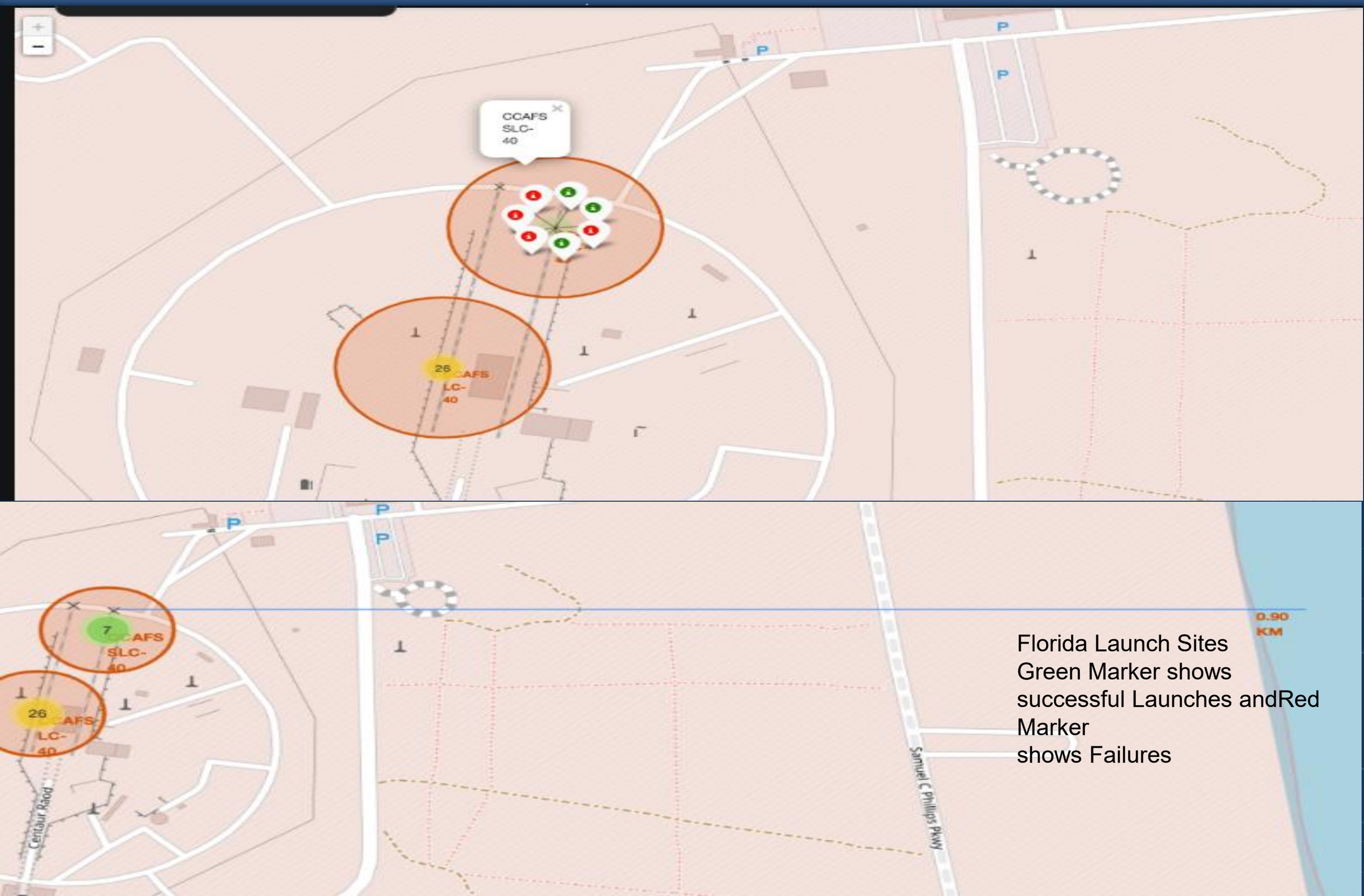
Query Explanation-
Using the GROUP BY clause on "Landing_Outcome" groups all records by their outcome type, and the WHERE clause with BETWEEN filters the results to only include dates from June 4, 2010 to March 20, 2017, while ORDER BY ... DESC sorts outcomes from most to least frequent.

INTERACTIVE MAP WITH FOLIUM

All launch sites global map markers

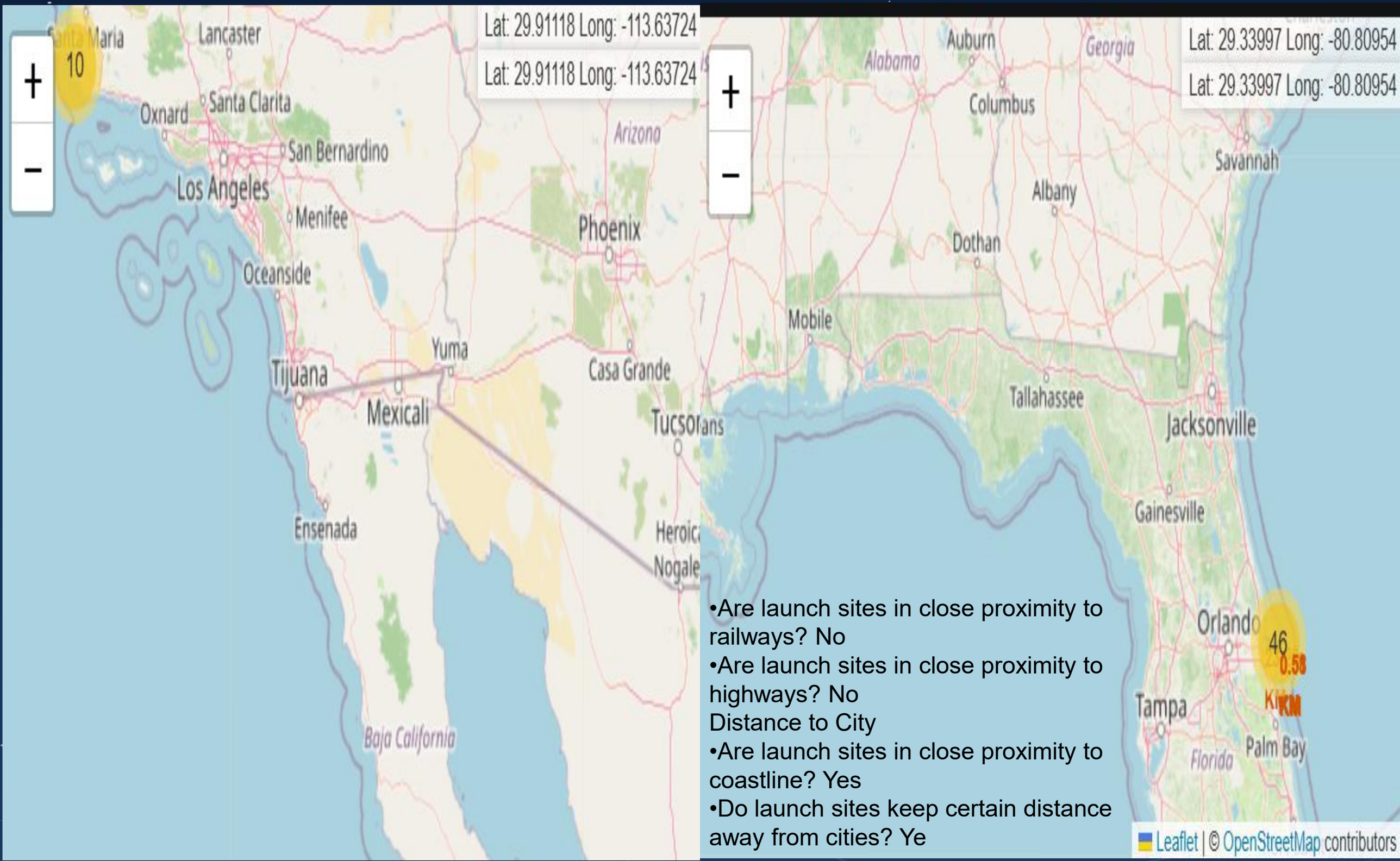
We can see that the SpaceX launch sites are in the United States of America coasts. Florida and California

Colour Labelled Markers



Florida Launch Sites
Green Marker shows successful Launches andRed Marker
shows Failures

Folium Map with distance calculated



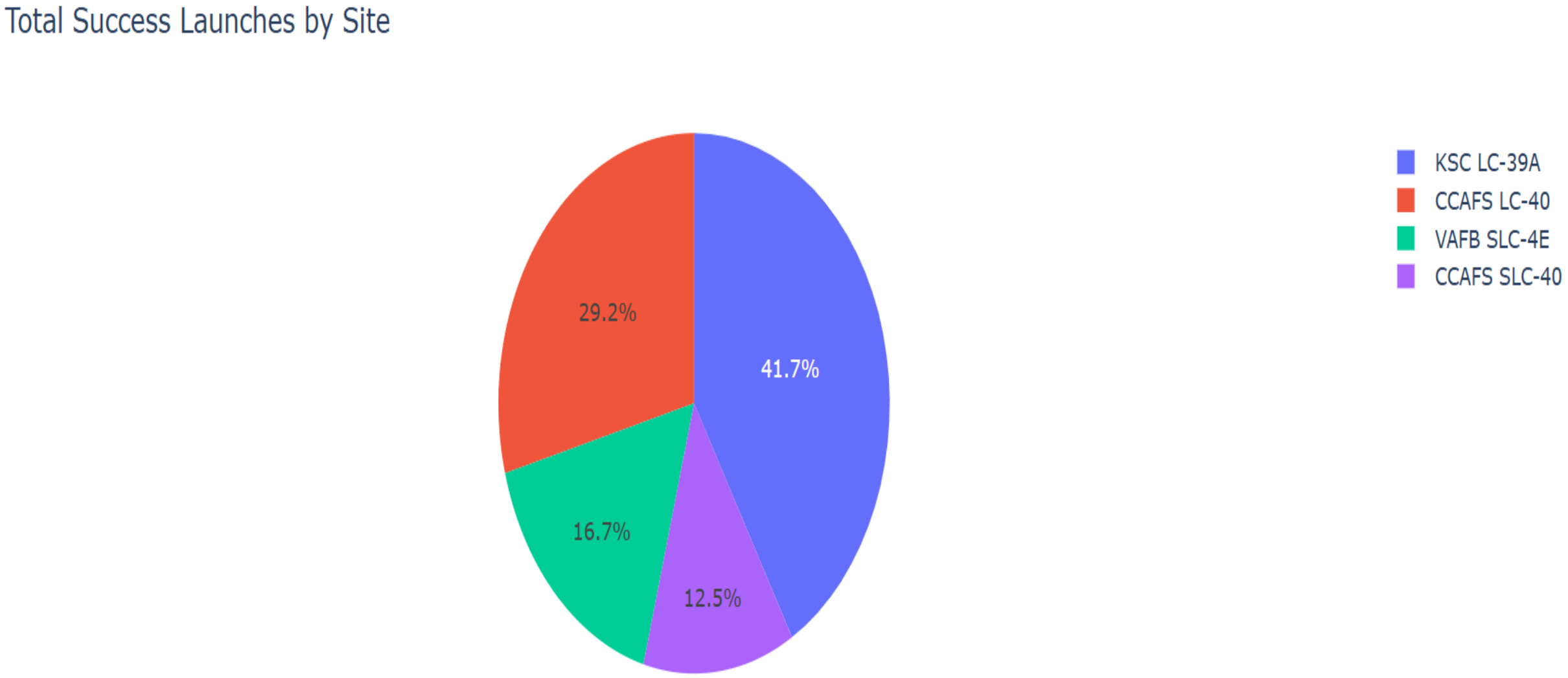
- Are launch sites in close proximity to railways? No
- Are launch sites in close proximity to highways? No

Distance to City

- Are launch sites in close proximity to coastline? Yes
- Do launch sites keep certain distance away from cities? Ye

DASHBORD WITH PLOTLY DASH

DASHBOARD–Pie chart showing the success percentage achieved by each launch site

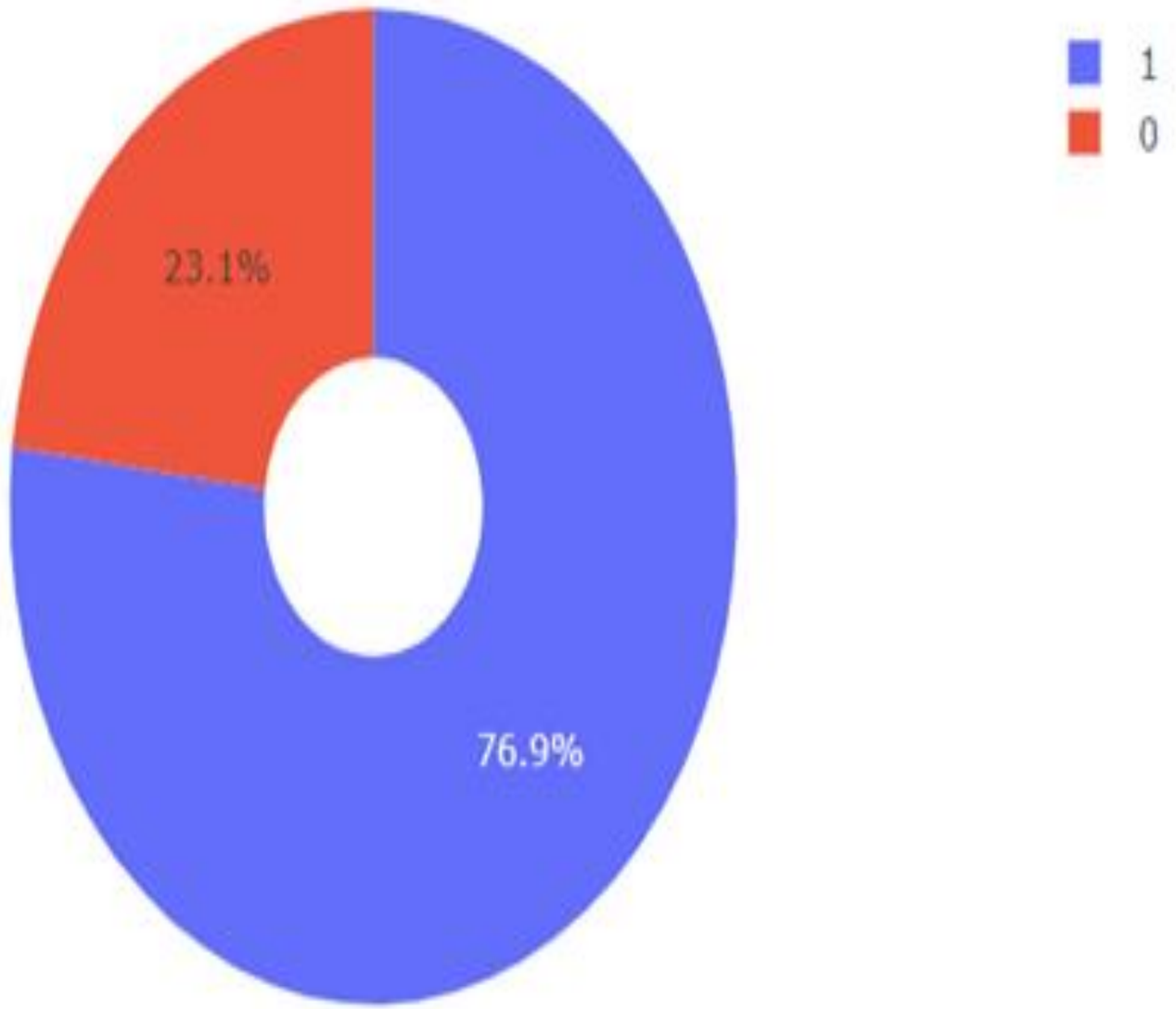


Payload range (Kg):

0

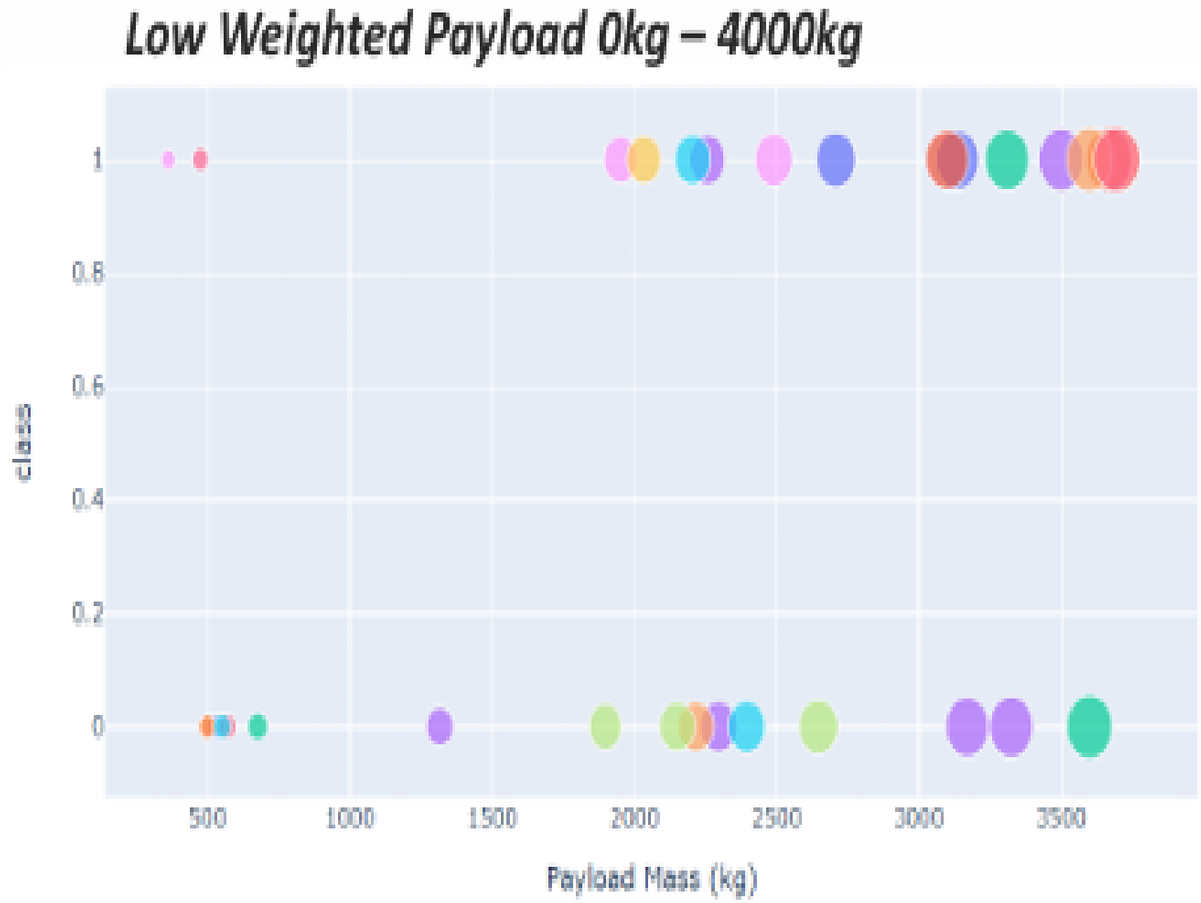
9600

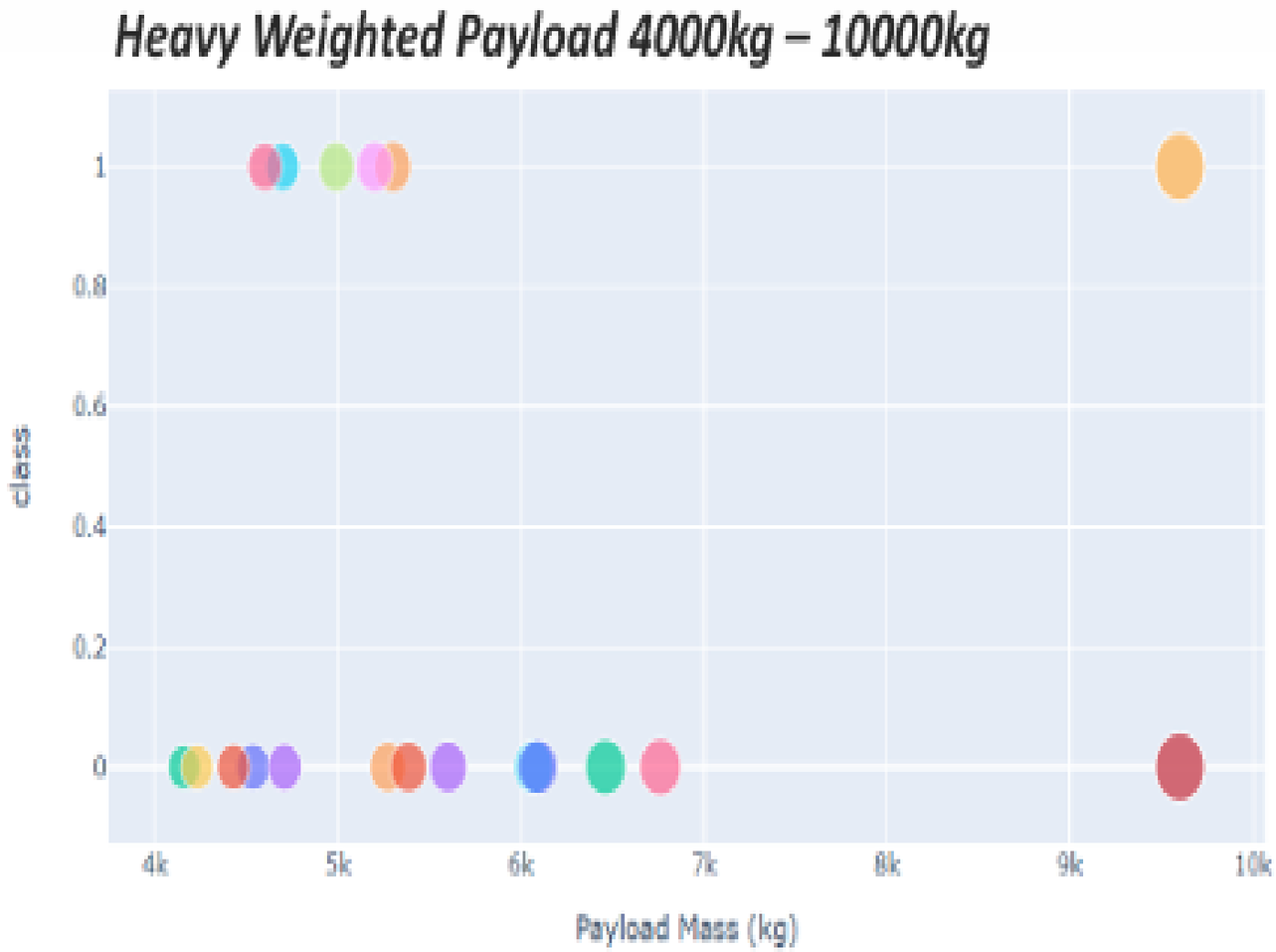
DASHBOARD–Pie chart for the launch site with highest launch success ratio



KSC LC-39A achieved a 76.9% success rate while getting a 23.1% failure rate

DASHBOARD–Payload vs. Launch Outcome scatter plot for all sites, with different payload selected in the range slider





We can see the success rates for low weighted payloads is higher than the heavy weighted payloads

Predictive analysis (Classification

Classification Accuracy using training data

As you can see our accuracy is extremely close but we do have a winner its down to decimal places! using this Function

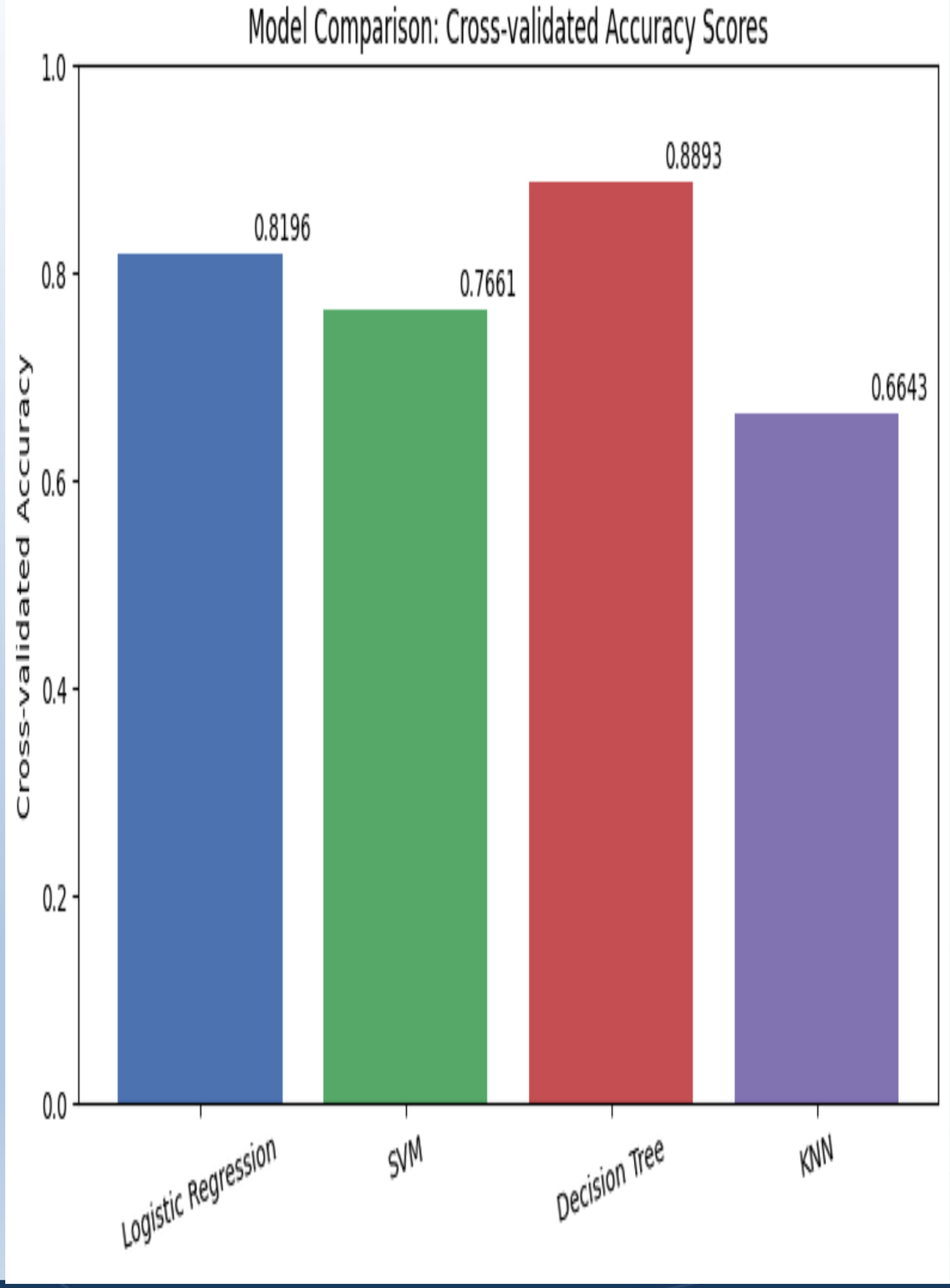
bestalgorithm = max(algorithms,key=algorithms.get)

s.no	Accuracy	Algorithm
0	0.6643	KNN
1	0.8893	Tree
2	0.7661	SVM
3	0.8196	Logistic regression

The tree algorithm wins!!

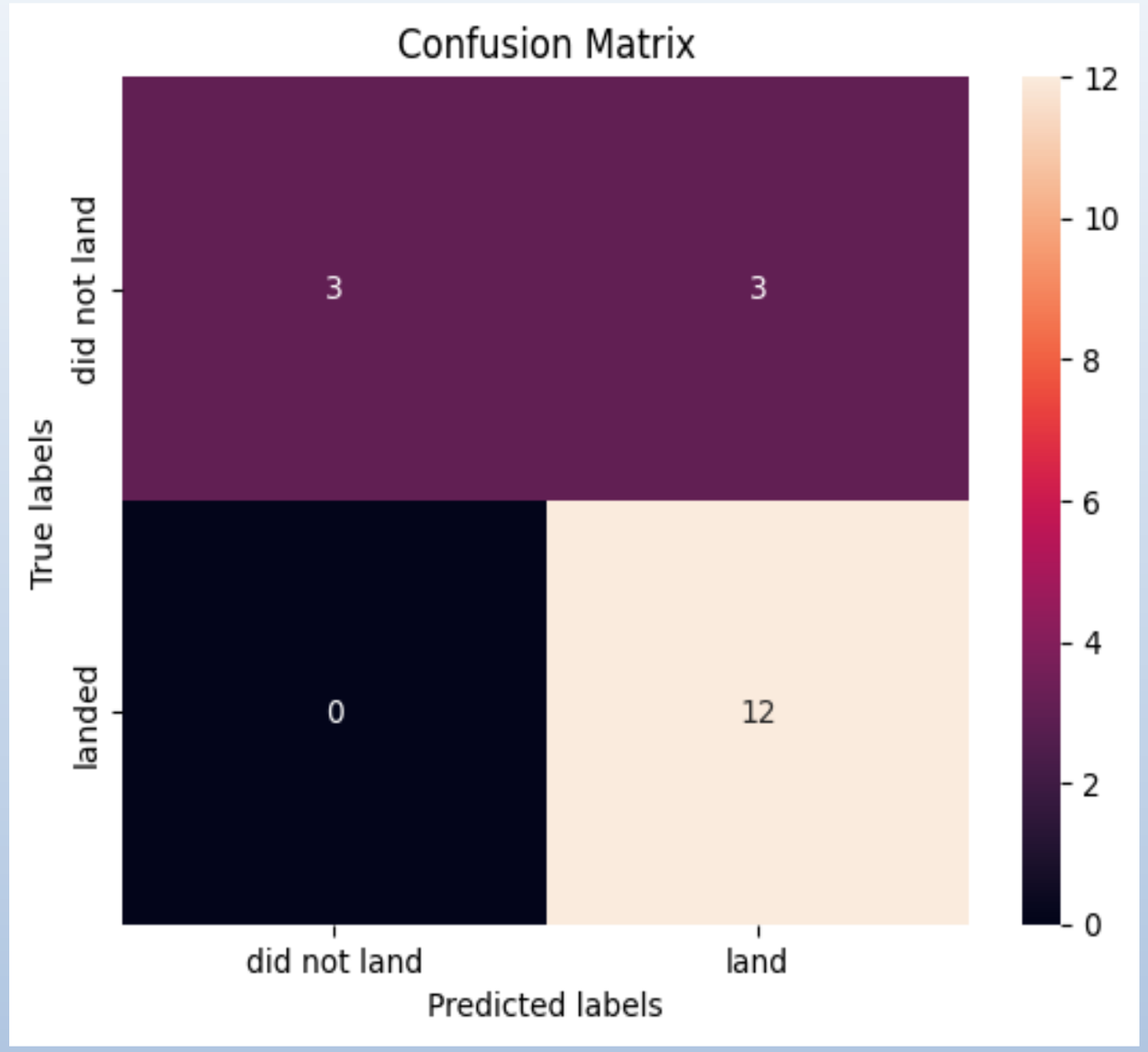
Best Aalgorithm is tree with a score of 0.8892857142857145
best para = {'criterion': ['gini', 'entropy'], 'splitter': ['best', 'random'], max_depth': [2*n for n in range(1,10)],max_features': ['auto', 'sqrt'], 'min_samples_leaf': [1, 2, 4],'min_samples_split': [2, 5, 10]}

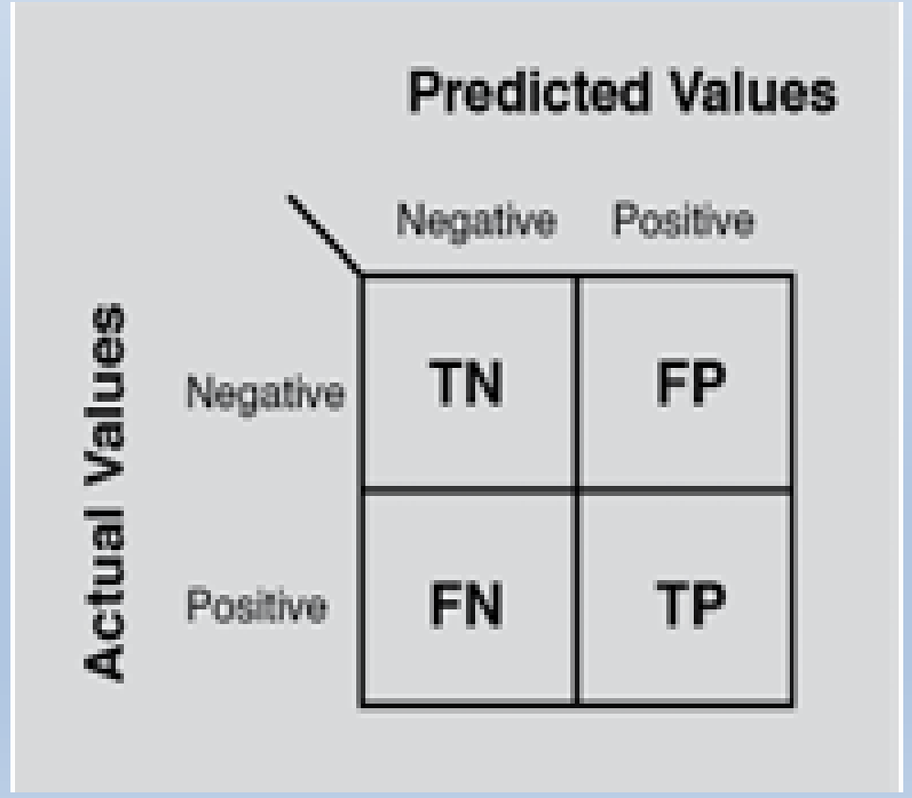
After selecting the best hyperparameters for the decision tree classifier using the validation data, we achieved 83.33% accuracy on the test data.



Confusion Matrix for the Tree

Examining the confusion matrix, we see that Tree can distinguish between the different classes. We see that the major problem is false Positives.





CONCLUSION

•The Decision Tree Classifier proves to be the most effective algorithm for this dataset

•Lighter payloads tend to yield better launch outcomes compared to heavier ones

•Launch success rates show a positive correlation with time, indicating SpaceX's success improves
as they gain more years of experience .

•KSC LC-39A stands out as the launch site with the highest number of successful missions

•The orbits GEO, HEO, SSO, and ES-L1 exhibit the highest success rates among all orbit types

•The Decision Tree Classifier proves to be the most effective algorithm for this dataset

Feature Importance Analysis

Introduction

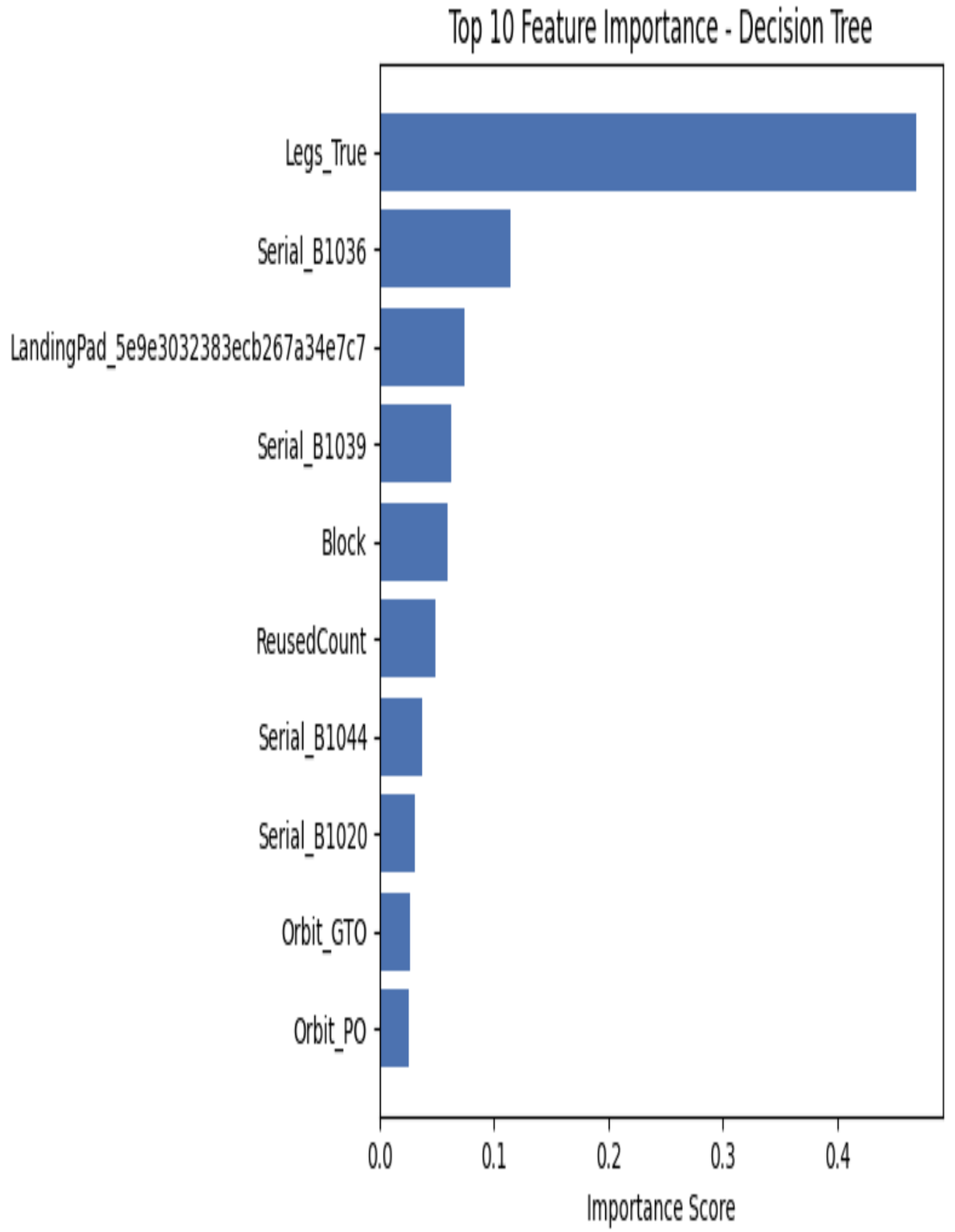
After training multiple classification models, I wanted to understand *why* the Decision Tree model was making its predictions — not just how accurate it was. So I extracted feature importance scores to see which factors (payload mass, orbit type, launch site, etc.) most strongly influenced launch success.

Implementation

- Extracted feature_importances_ from the best Decision Tree estimator.

- Plotted a horizontal bar chart ranking each feature by importance
- Used this to validate/support the key findings (e.g., payload mass and orbit type mattering)

THIS PART IS CREATED BY ME



ROC Curve Comparison

Introduction

Accuracy alone doesn't tell the full story of a classifier's performance, especially with imbalanced data. So I plotted ROC curves and calculated AUC scores for all four models to compare their true positive vs false positive trade-offs more rigorously.

Implementation

•Computed predicted probabilities for each model

•Plotted ROC curves for Logistic Regression, SVM, Decision Tree, and KNN on the same graph

•Compared AUC scores to further validate the "best model" conclusion

•THIS PART IS CREATED BY ME